<!--Análisis y desarrollo de Software-->

Workflow Git + Github {

<Por="Macarron"/>

}





Introducción {

¿Qué es un Workflow en el Contexto de Desarrollo de Software?



Colaborar de manera eficiente al crear, modificar o mejorar software.

Establece cómo se manejan las tareas, los cambios al código y las revisiones para entregar un producto de calidad.

}

Tecnologías y herramientas {



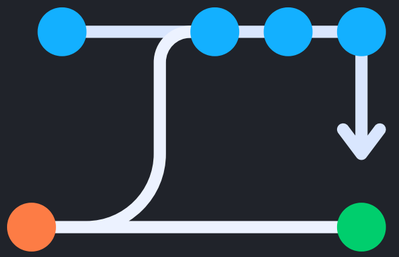
GIT

SISTEMA DE CONTROL
DE VERSIONES



GitHub

HERRAMIENTA PARA
GESTIONAR VERSIONES



Git Flow

MÉTODOLOGIA DE
TRABAJO EN EQUIPO

}

¿Qué es Git? {

- Registrar cambios realizados en un proyecto a lo largo del tiempo.
- Colaborar con otros programadores de manera eficiente.
- Restaurar versiones anteriores si algo sale mal.



git

Imaginen un historial de versiones para tu proyecto donde puedes regresar a un archivo o estado anterior.

}

GitHub {

- Ofrece un lugar centralizado para almacenar repositorios Git.
- Facilita la colaboración y la comunicación entre equipos de desarrollo.
- Integra herramientas para revisiones de código, issues, y automatizaciones.



}

Conceptos clave {



01

Branch (Rama)

Una rama es una línea paralela de desarrollo en Git. Permite trabajar en nuevas funcionalidades o correcciones sin afectar el código principal.

`main`

`feature`

02

Clonar un repositorio

Clonar significa descargar una copia de todo el repositorio (incluyendo su historial de versiones) a tu máquina local, para trabajar en él.

```
git clone <url>
```

03

Manejo de cambios

Son las acciones para registrar y enviar tus modificaciones al proyecto:

```
git add <archivo>
```

```
git commit -m "Descripción"
```

```
git push
```

04

Pull request

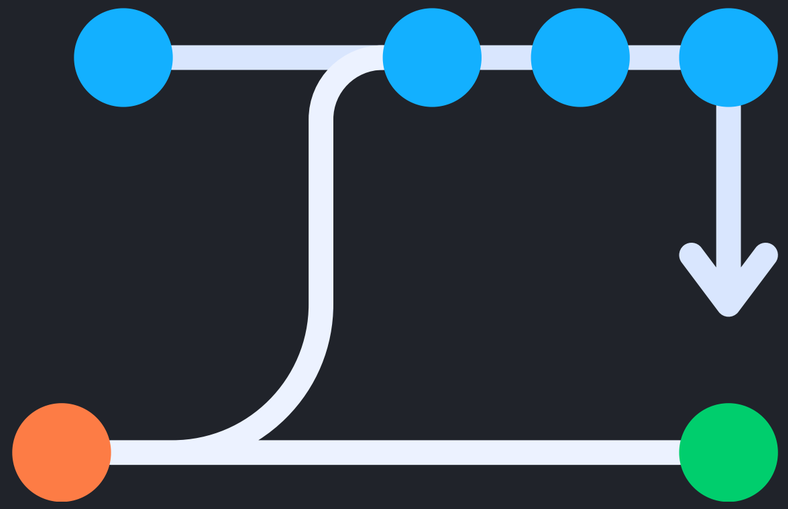
Un Pull Request (PR) en GitHub es una solicitud para fusionar cambios realizados en una rama (como feature/nueva-funcion) con otra (como main).

- Comparación de código.
- Revisión de compañeros.
- Aprobación antes de fusionar.

}

Git Flow {

Git Flow es una estrategia de trabajo que organiza el desarrollo de software con ramas (branches) específicas. Es ideal para proyectos que requieren control de versiones claro y colaboraciones organizadas.



}

Estructura {



}

Ramas {

Principales:

Master — Rama principal o de producción, contiene la versión estable.

Develop — Rama para la creación de nuevas modificaciones o funcionalidades del proyecto

Temporales:

Feature — Rama creada para corrección de parches de bug en producción estas son solucionadas de manera rápida

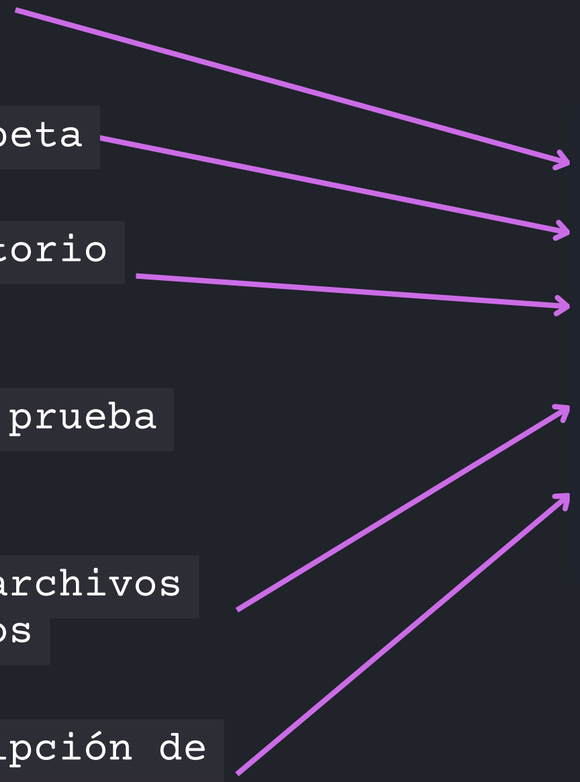
HotFix — Es la rama principal de desarrollo donde se integran nuevas funciones antes de pasar a producción.

}

Inicialización del Repositorio por el Administrador

Crear el repo en local

1. Creamos la carpeta
2. Ingresamos a la carpeta
3. Iniciamos un repositorio de git
4. Creamos archivos de prueba (index.html)
5. Añadimos todos los archivos creados a los cambios
6. Escribimos la descripción de lo que añadimos



```
mkdir proyecto-colaborativo
cd proyecto-colaborativo
git init
git add .
git commit -m "Primer commit"
```

Subirlo a GitHub

1. Conectamos el repo local al que creamos en github
2. Nos ubicamos en la rama Main
3. Empujamos los cambios a github

```
git remote add origin <URL-del-repo>
git branch -M main
git push -u origin main
```

Creación de ramas con Git Flow

1. Nos cambiamos a la rama de develop

```
git checkout -b develop
```

2. Empujamos la rama creada al repo en gitHub

```
git push -u origin develop
```

3. Nos movemos a la rama de develop

```
git checkout develop
```

4. Crear una rama de feature para empezar a trabajar

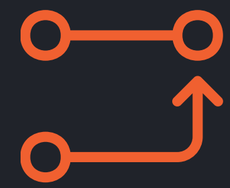
```
git checkout -b feature/inicializacion-html
```

Creación de ramas con Git Flow

Realizamos algún cambio al archivo de index y subimos los cambios a la rama que acabamos de crear

```
git add index.html
git commit -m "Inicialización de estructura HTML"
git push -u origin feature/inicializacion-html
```

Luego de esto creamos un Pull Request en GitHub a la rama develop



Segundo colaborador

1. Clonamos el repositorio con el link
2. Ingresamos al proyecto creado

```
git clone <URL-del-repo>
cd proyecto-colaborativo
```

3. Nos movemos a la rama de develop

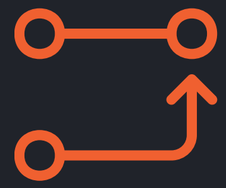
```
git checkout develop
```

4. Nos aseguramos de traer los cambios de develop

```
git pull origin develop
```

Segundo colaborador

Revisamos el PR, lo aprobamos y fusionamos a develop desde GitHub



1. Crear una nueva rama feature/estilos-css para trabajar

```
git checkout -b feature/estilos-css
```

2. Creamos y agregamos el archivo styles.css

```
git add styles.css
git commit -m "Agregar estilos básicos con CSS"
git push -u origin feature/estilos-css
```

Abrimos un nuevo Pull Request en GitHub hacia la rama develop

Integración Continua y Fusión de Cambios

1. El administrador o el equipo integra las ramas feature/... revisadas en la rama develop



```
git checkout develop
git merge feature/inicializacion-html
git merge feature/estilos-css
git push origin develop
```

2. Una vez que los cambios en develop son estables, integrarlos a la rama main para lanzar una versión oficial



```
git checkout main
git merge develop
git push origin main
```

¿Qué es un .gitignore? {

Un archivo .gitignore es un archivo de configuración que le indica a Git qué archivos y directorios deben ser excluidos del control de versiones.

Esto es útil para evitar subir archivos temporales, locales o irrelevantes al repositorio.



Normalmente se crea automáticamente cuando se crea un proyecto con React o Express.

}

<!--Análisis y desarrollo de Software-->

Gracias {

<Por="Macarron"/>

}

